Assignment 10

Analysis of Brownian Motion and Active Brownian Motion

Submitted by

Sanjeev Kumawat
`ph25mscst11026`

Instructor
Anupam Gupta

Date of Submission

April 8, 2026

Contents

1 Introduction **2**

2 Initial and Final Particle Position Snapshots **2**

3 Mean Square Displacement **4**
3.1 Discussion 5

4 Qualitative Analysis of Motion **6**

5 Linear Fitting: α and C values **6**
5.1 Extracted Results 6

6 Physical Parameters (D_t, v, D_r) **7**

7 Conclusion **8**

A Python Code **8**
A.1 MSD Calculation 10
A.2 Calculation of α and c 10
A.3 Calculation of D_t, v and D_r 11

1 Introduction

In this assignment, we analyze the motion of particles using different datasets by studying their mean square displacement (MSD). MSD helps us understand how particles spread over time and is a key quantity to identify the nature of motion.
Different types of motion show different MSD behavior. In Brownian motion, MSD increases linearly with time, while in ballistic motion it increases quadratically. Active Brownian motion is more complex, showing different behavior at different time scales due to self-propulsion and rotational effects.

By plotting particle positions and analyzing the MSD (especially using log–log plots), we determine the type of motion in each dataset and extract important physical parameters such as diffusion coefficient, velocity, and rotational diffusion.

2 Initial and Final Particle Position Snapshots

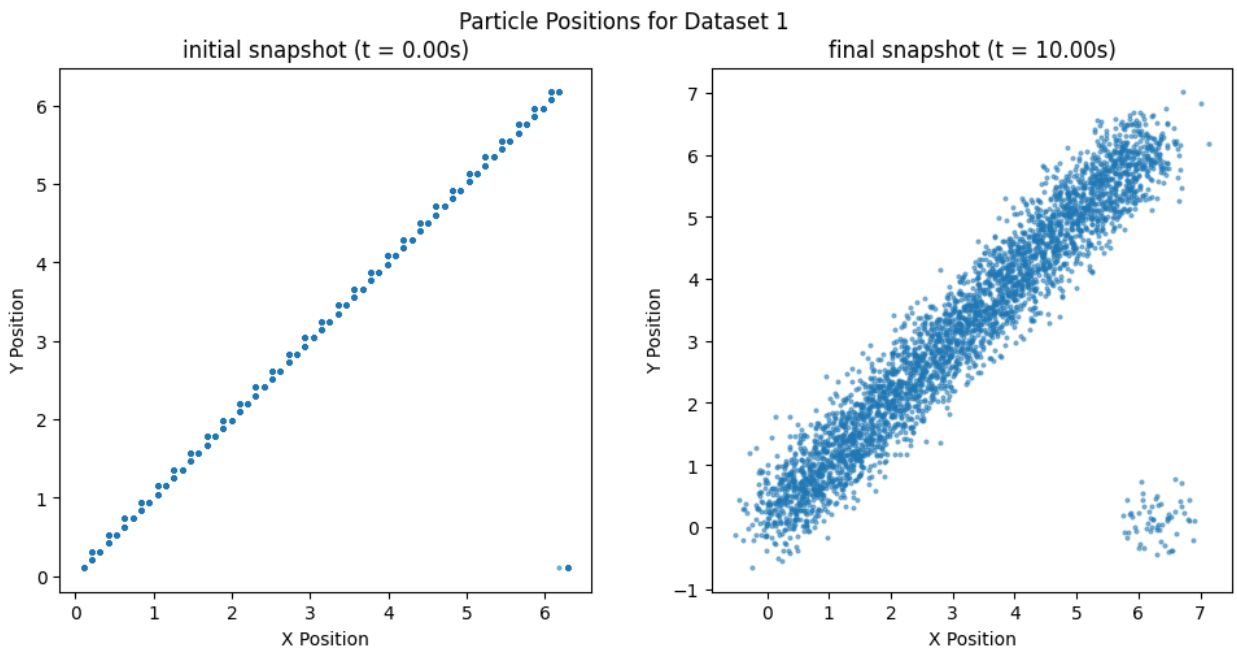

Figure 1

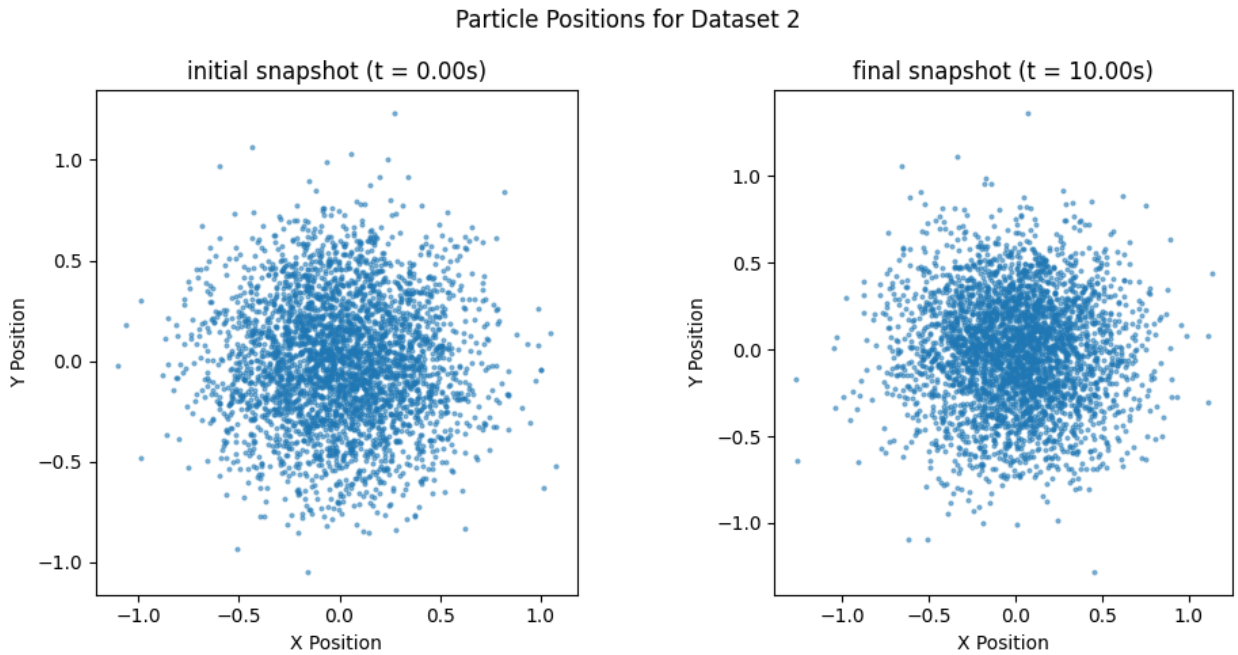

Figure 2

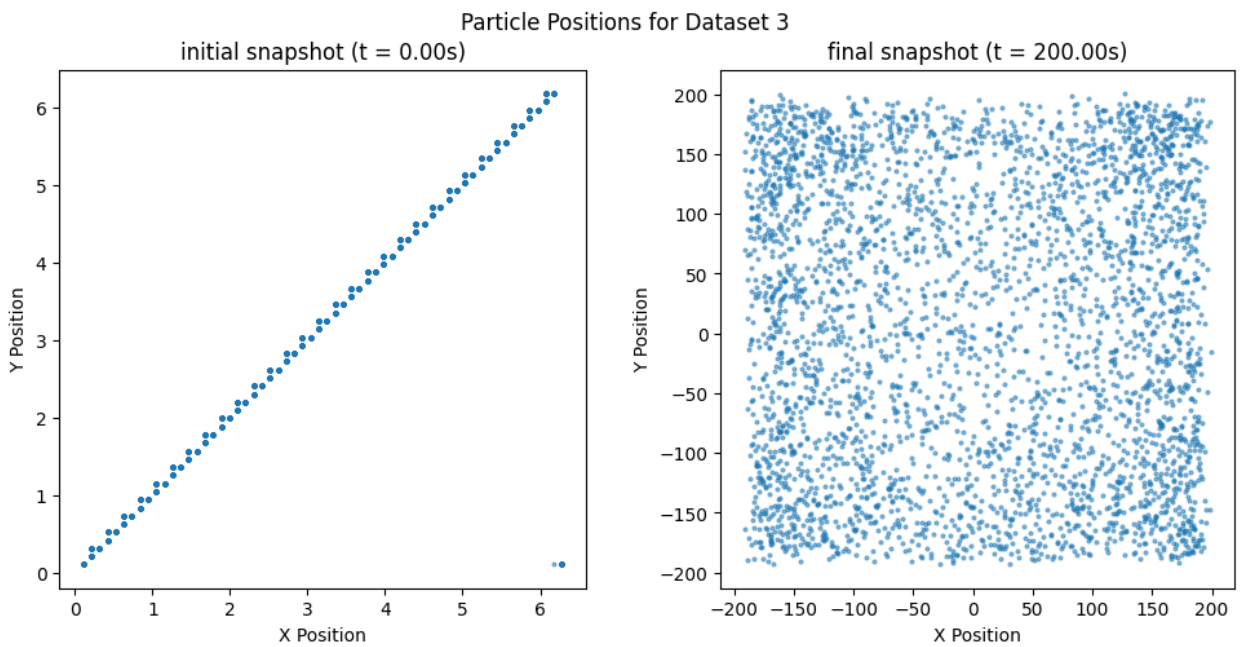

Figure 3

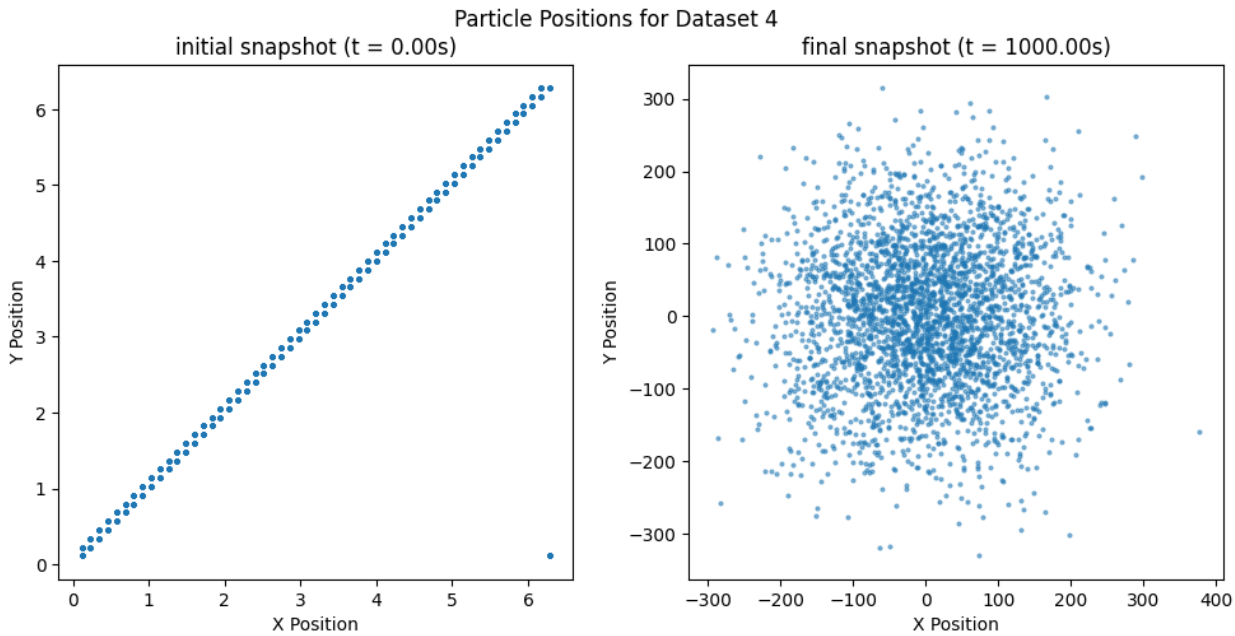

Figure 4

The particle positions at the initial and final time steps were plotted for all datasets. At the initial time, particles are more localized, while at later times they spread out due to motion.
Dataset 1 shows random spreading typical of Brownian motion. Dataset 3 shows more directed motion, indicating ballistic behavior. Dataset 4 displays a combination of spreading patterns, suggesting active motion. These plots give a clear visual understanding of how particle dynamics evolve over time.

3 Mean Square Displacement

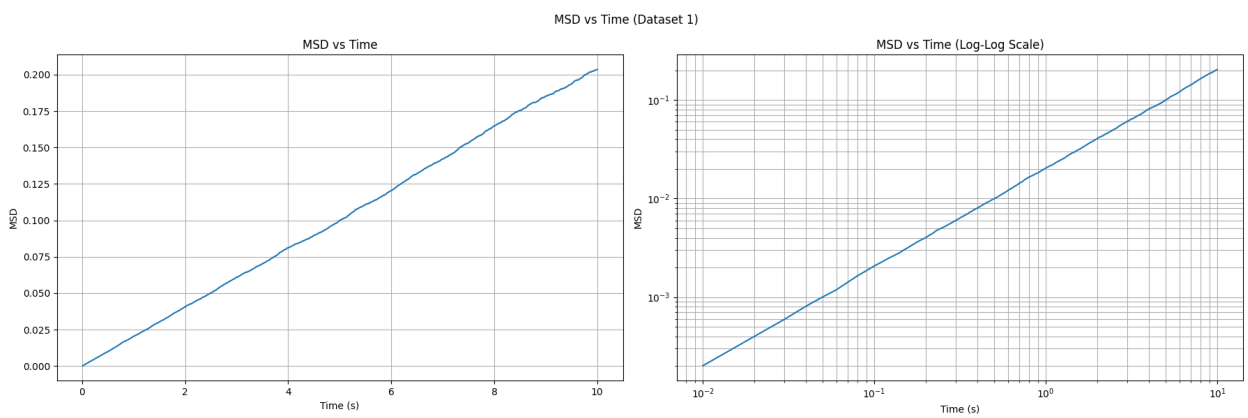

Figure 5

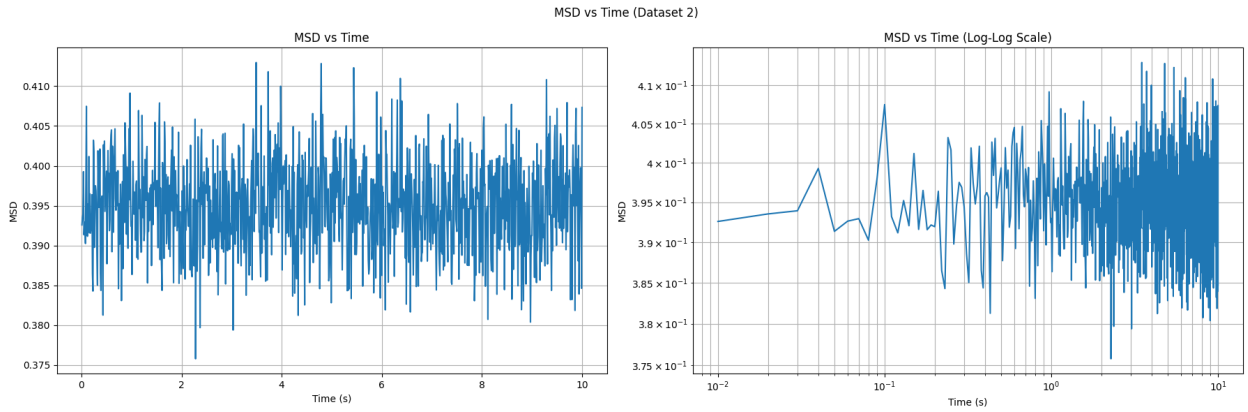

Figure 6

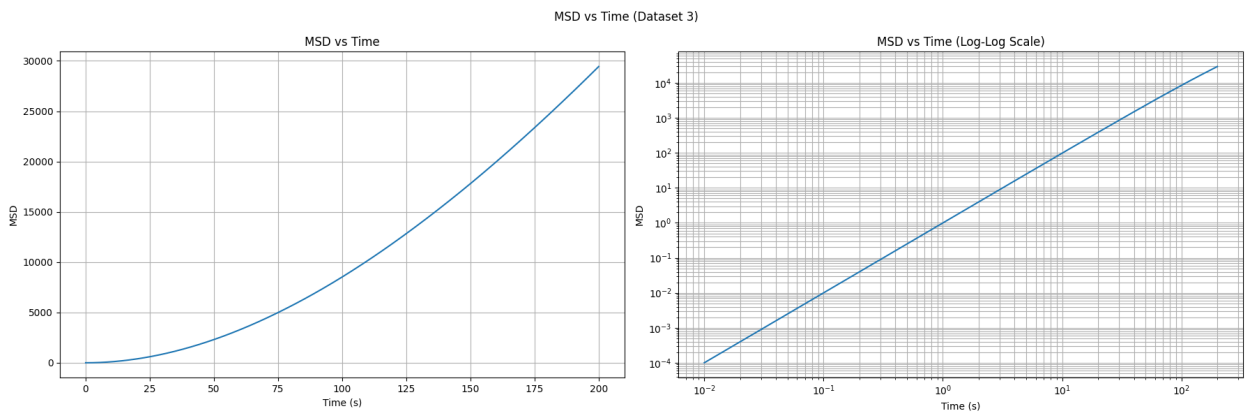

Figure 7

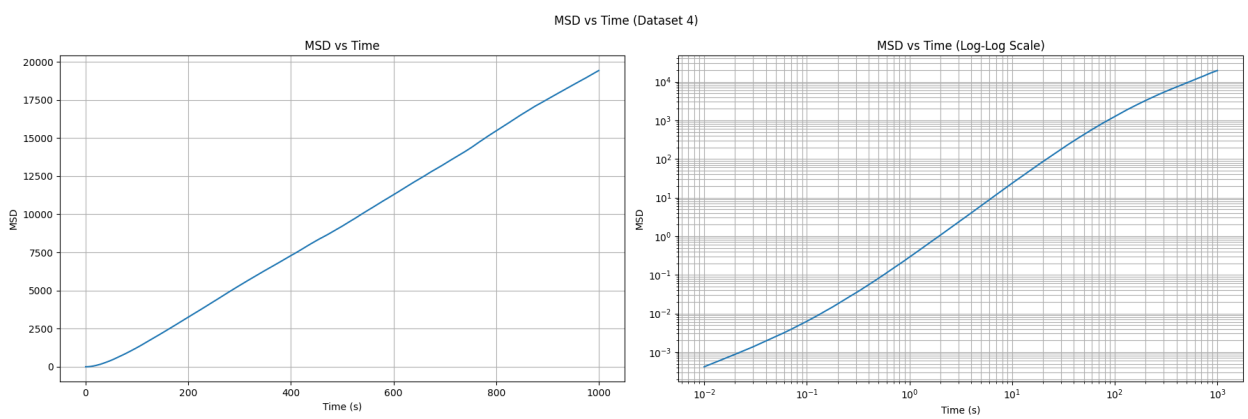

Figure 8

3.1 Discussion

The mean square displacement (MSD) was calculated to measure how particle positions change over time. It represents the average squared displacement from the initial position.

MSD was plotted against time in both normal and log–log scales. The log–log plot helps identify the type of motion based on its slope.

4 Qualitative Analysis of Motion

From the log–log plots of MSD vs time, the nature of motion for each dataset was identified based on the slope.
Dataset 1 shows a slope close to 1, indicating Brownian motion. Dataset 3 has a slope close to 2, which corresponds to ballistic motion. Dataset 4 shows a changing slope, indicating a transition between diffusive and ballistic behavior, characteristic of active Brownian motion. Dataset 2 does not show consistent behavior, suggesting possible issues with the data.

5 Linear Fitting: α and C values

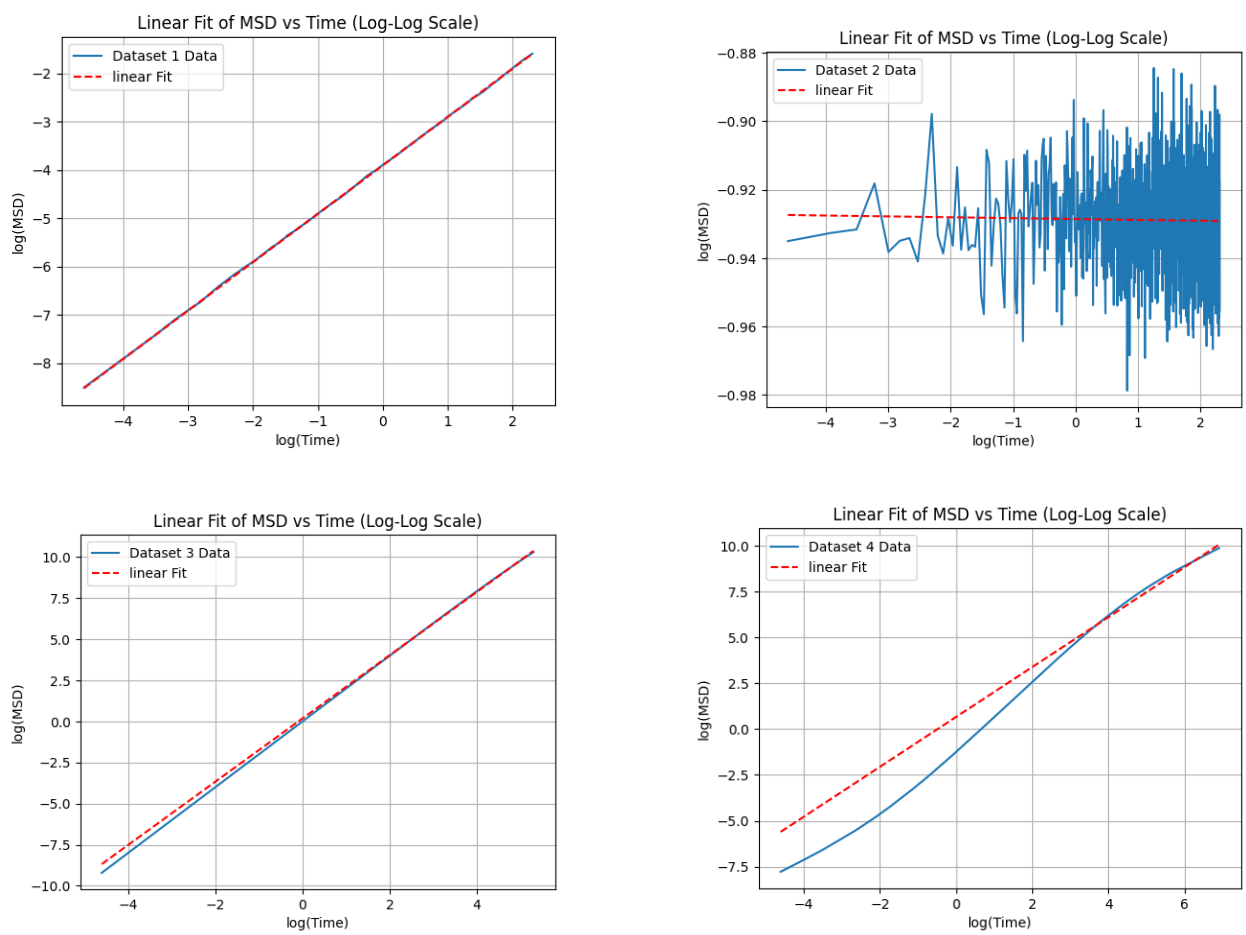

Figure 9

5.1 Extracted Results

```
Dataset 0:
a = 1.0031
```

```
C = 0.0202

Dataset 1:
a = -0.0003
C = 0.3951

Dataset 2:
a = 1.9225
C = 1.1924

Dataset 3:
a = 1.3609
C = 1.9393
```

The mean square displacement (MSD) data was fitted using the relation

$$\Delta(t) = C\, t^{\alpha}.$$

To linearize the data, a log–log transformation was used:

$$\log \Delta(t) = \log C + \alpha \log t.$$

A linear fit was then applied to the transformed data to extract the slope α and the intercept $\log C$. The exponent α characterizes the nature of the motion, while the constant C is obtained from the intercept.
For Dataset 4, different time regions were analyzed separately due to a change in dynamical behavior.

6 Physical Parameters (D_t, v, D_r)

```
D_t = 0.0050
v = 1.0920
D_r = 0.1175
```

The physical parameters were obtained from the fitted MSD relations for different datasets.
For Dataset 1 (Brownian motion), the translational diffusion coefficient was calculated using

$$D_t = \frac{C}{4}.$$

For Dataset 3 (ballistic motion), the velocity was obtained from

$$v = \sqrt{C}.$$

For Dataset 4 (active Brownian motion), the large-time region was considered, and the rotational diffusion coefficient was calculated using

$$D_r = \frac{2v^2}{\text{slope}}.$$

These parameters provide a quantitative description of the different types of particle motion observed in the system.

7 Conclusion

The analysis of the MSD data successfully distinguishes between different types of particle motion. Brownian motion exhibits diffusive behavior with $\alpha \approx 1$, while ballistic motion shows $\alpha \approx 2$, indicating directed motion. Active Brownian motion demonstrates a transition from ballistic to diffusive behavior over time.
The extracted parameters, including the translational diffusion coefficient, velocity, and rotational diffusion coefficient, provide a quantitative understanding of the underlying dynamics. Overall, the results highlight how MSD analysis effectively characterizes different regimes of motion in stochastic systems.

A Python Code

```
#importing required libraries
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit
import os

#function for plotting the initial and final snapshot of the given dataset
def plot_particle_positions(base_path, positions, N, dataset_name):
    fig, axes = plt.subplots(1, 2, figsize=(10, 5))
    plt.suptitle(f'Particle Positions for {dataset_name}')
    titles = [f'initial snapshot', f'final snapshot']

    for ax, title, i in zip(axes, titles, positions):
        file_path = f"{base_path}{i}"

        if not os.path.exists(file_path):
            print(f"File not found: {file_path}")
            continue

        data = np.fromfile(file_path, dtype=np.float64)
        reshaped_data = data.reshape(N, 2)

        x = reshaped_data[:, 0]
        y = reshaped_data[:, 1]

        ax.scatter(x, y, s=4, alpha=0.5)
        ax.set_title(f"{title} (t = {i*0.001:.2f}s)")
        ax.set_xlabel('X Position')
        ax.set_ylabel('Y Position')
        ax.set_aspect('equal')

    plt.tight_layout()
    plt.show()

#mean square displacment for the given dataset
def mean_square_displacement(base_path, positions, N, dataset_name):
    dt = 0.001 #seconds
    msd_values = []
    time_diff = []
```

```
    start, end = positions

    #initial positions
    file_path = f"{base_path}{start}"
    initial_data = np.fromfile(file_path, dtype=np.float64)
    reshaped_data = initial_data.reshape(N, 2)
    xi = reshaped_data[:, 0]
    yi = reshaped_data[:, 1]

    for i in range(start+10, end+1, 10):
        file_path = f"{base_path}{i}"

        if not os.path.exists(file_path):
            print(f"File not found: {file_path}")
            continue

        data = np.fromfile(file_path, dtype=np.float64)
        reshaped_data = data.reshape(N, 2)
        xf = reshaped_data[:, 0]
        yf = reshaped_data[:, 1]

        msd = np.mean((xf - xi)**2 + (yf -  yi)**2)
        delta_t = dt * i

        msd_values.append(msd)
        time_diff.append(delta_t)


    fig, ax = plt.subplots(1, 2, figsize=(18, 6))
    plt.suptitle(f'MSD vs Time ({dataset_name})')

    ax[0].plot(time_diff, msd_values)
    ax[0].set_title('MSD vs Time')
    ax[0].set_xlabel('Time (s)')
    ax[0].set_ylabel('MSD')
    ax[0].grid(True)

    ax[1].loglog(time_diff, msd_values)
    ax[1].set_title('MSD vs Time (Log-Log Scale)')
    ax[1].set_xlabel('Time (s)')
    ax[1].set_ylabel('MSD')
    ax[1].grid(True, which="both")

    plt.tight_layout()
    plt.show()
    return msd_values, time_diff

#dataset 1
base_path_1 = rf'/home/Interstellar/Desktop/Computational Methods/
   Assignment-10(Brownian Motion)/Active_BrownianParticle_DataSet/
   Finalized_data/DataSet1/part/POSITION'

#dataset 2
base_path_2 = rf'/home/Interstellar/Desktop/Computational Methods/
```

```
    Assignment-10(Brownian Motion)/Active_BrownianParticle_DataSet/
    Finalized_data/DataSet2/part/NOISE_tra'

#dataset 3
base_path_3 = rf'/home/Interstellar/Desktop/Computational Methods/
    Assignment-10(Brownian Motion)/Active_BrownianParticle_DataSet/
    Finalized_data/DataSet3/part/POSITION'

#dataset 4
base_path_4 = rf"/home/Interstellar/Desktop/Computational Methods/
    Assignment-10(Brownian Motion)/Active_BrownianParticle_DataSet/
    Finalized_data/DataSet4/part/POSITION"

#total number of the given particals
N1, N2, N3, N4 = 3517, 3517, 3517, 3000

#final snapshots
fs1, fs2, fs3, fs4 =  10000, 10000, 200000, 1000000

plot_particle_positions(base_path_1, [0, fs1], N1, "Dataset 1")
plot_particle_positions(base_path_2, [0, fs2], N2, "Dataset 2")
plot_particle_positions(base_path_3, [0, fs3], N3, 'Dataset 3')
plot_particle_positions(base_path_4, [0, fs4], N4, "Dataset 4")
```

A.1 MSD Calculation

```
msd_values1, time_diff1 = mean_square_displacement(base_path_1, [0, fs1],
    N1, "Dataset 1")
msd_values2, time_diff2 = mean_square_displacement(base_path_2, [0, fs2],
    N2, "Dataset 2")
msd_values3, time_diff3 = mean_square_displacement(base_path_3, [0, fs3],
    N3, "Dataset 3")
msd_values4, time_diff4 = mean_square_displacement(base_path_4, [0, fs4],
    N4, "Dataset 4")
```

A.2 Calculation of α and c

```
def linear_fit(t, m, c):
    fit_line = m*t + c
    return fit_line

#linear curve fit using scipy
popt1, _ = curve_fit(linear_fit, np.log(time_diff1), np.log(msd_values1))
popt2, _ = curve_fit(linear_fit, np.log(time_diff2), np.log(msd_values2))
popt3, _ = curve_fit(linear_fit, np.log(time_diff3), np.log(msd_values3))
popt4, _ = curve_fit(linear_fit, np.log(time_diff4), np.log(msd_values4))



#extract parameters
params = [popt1, popt2, popt3, popt4]
```

```
t = [np.log(time_diff1), np.log(time_diff2), np.log(time_diff3), np.log(
    time_diff4)]
msd = [np.log(msd_values1), np.log(msd_values2), np.log(msd_values3), np.
    log(msd_values4)]

for i, (m, c) in enumerate(params, start=0):
    alpha = m
    C = np.exp(c)

    print(f"Dataset {i}:")
    print(f"\u03B1 = {alpha:.4f}")
    print(f"C = {C:.4f}")
    print()

for i in range(4):
    plt.plot(t[i], msd[i], label=f"Dataset {i+1} Data")
    plt.plot(t[i], linear_fit(t[i], params[i][0], params[i][1]),
             '--', label=f"linear Fit", color='red')

    plt.title("Linear Fit of MSD vs Time (Log-Log Scale)")
    plt.xlabel("log(Time)")
    plt.ylabel("log(MSD)")
    plt.legend()
    plt.grid(True, which="both")
    plt.show()
    print(i)
```

A.3 Calculation of D_t, v and D_r

```
#Dataset 1: Brownian Motion
D_t = np.exp(popt1[1])/4

#Dataset 2:

#Dataset 3: Ballistic Motion
v = np.sqrt(np.exp(popt3[1]))

#Dataset 4: Active Brownian Motion
t_large = np.array(time_diff4[7000:])
msd_large = np.array(msd_values4[7000:])

popt_large, _ = curve_fit(linear_fit, t_large, msd_large)
slope_large = popt_large[0]

Dr = (2 * v**2) / slope_large

print(f"D_t = {D_t:.4f}")
print(f"v = {v:.4f}")
print(f"D_r = {Dr:.4f}")


plt.plot(t_large, msd_large, label='Data')
plt.plot(t_large, linear_fit(t_large, *popt_large), '--', label='Fit')
```

```
plt.xlabel('Time (s)')
plt.ylabel('MSD')
plt.title('Large Time Fit (Dataset 4)')
plt.legend()
plt.grid()
plt.show()
```

References

[1] Sanjeev K, "Python code used in this report," GitHub repository, 2026. Available at: https://github.com/arsanjeevk/Computational_Methods/tree/main/Assignment-10(Brownian%20Motion) [Accessed: 8 April 2026].